

Technology Stack and Usage Document

Recruitment Assist full-stack AI recruitment application

Project Overview

Recruitment Assist is a full-stack AI-powered recruitment platform. It manages job descriptions, resumes, candidate comparison, interview scheduling, communication drafts, dashboards, reports, and recruiter assistance.

Core Stack



Application Flow

1. User works in the React frontend.
2. frontend/src/api.js sends requests to Flask and attaches the session token.
3. Flask routes call services, workflows, or agents.
4. MongoDB stores users, jobs, candidates, comparisons, interviews, audit logs, and agent runs.
5. OpenAI and LangGraph power extraction, screening, matching, summaries, and recruiter assistance.

Frontend Technologies

Where each technology is used and why

React 18	Where: frontend/src/App.jsx, frontend/src/pages/*, frontend/src/components/* Why: Builds the single-page application UI with reusable components.
React DOM	Where: frontend/src/index.js Why: Mounts the React app into the browser DOM.
React Router DOM	Where: frontend/src/App.jsx and files using Link/useNavigate/useParams Why: Handles routes like /dashboard, /jobs, /analyze, /talent, /interviews, and /profile.
Create React App / react-scripts	Where: frontend/package.json Why: Provides the dev server, build command, test runner, and standard React tooling.
CSS	Where: frontend/src/styles/*.css and frontend/src/index.css Why: Controls layout, branding, dashboard styling, forms, reports, and responsive UI.
Flatpickr	Where: frontend/src/pages/Dashboard.jsx and frontend/src/styles/dashboard.css Why: Provides dashboard date picker controls.
Fetch API	Where: frontend/src/api.js Why: Sends HTTP requests from React to Flask APIs.
localStorage / sessionStorage	Where: frontend/src/api.js Why: Stores session tokens and small frontend caches.
Tailwind config	Where: frontend/tailwind.config.js Why: Defines design tokens; the current UI mainly uses custom CSS files.

Backend Technologies

Where each technology is used and why

Python	Where: backend/**/* Why: Main backend programming language.
Flask	Where: backend/app.py and backend/routes/* Why: Provides API routing, request handling, sessions, redirects, and JSON responses.
Flask Blueprints	Where: backend/app.py and backend/routes/* Why: Organizes APIs by feature: auth, dashboard, jobs, candidates, matching, interviews, reports, audit, and agentic routes.
Flask-CORS	Where: backend/app.py Why: Allows React localhost frontend requests to Flask /api/* endpoints during development.
Werkzeug	Where: backend/database.py and backend/services/* Why: Handles password hashing and secure uploaded filenames.
python-dotenv	Where: backend/app.py and backend/agents/openai_config.py Why: Loads .env settings such as MongoDB URI, OpenAI key, Flask config, SMTP, and Twilio values.
Pydantic	Where: backend/schemas/agentic.py Why: Validates structured agentic API payloads.
pytest	Where: backend/tests/test_security_hardening.py Why: Runs backend security and regression tests.

Database and Persistence

Where each technology is used and why

MongoDB / MongoDB Atlas	Where: backend/database.py Why: Stores users, job descriptions, candidates, comparisons, interviews, audit logs, agent runs, tokens, and counters.
PyMongo	Where: backend/database.py Why: Connects to MongoDB, creates indexes, queries collections, and updates documents.
MongoDB indexes	Where: backend/database.py Why: Improves lookups and enforces uniqueness for usernames, job IDs, candidate IDs, comparisons, interviews, and tokens.
Integer counters	Where: backend/database.py Why: Keeps simple numeric IDs in URLs and payloads while using MongoDB documents internally.

AI and Agentic Technologies

Where each technology is used and why

OpenAI Python SDK	Where: backend/app/llm_extraction.py and backend/agents/*.py Why: Performs extraction, resume/JD parsing, matching, ranking, summaries, interview content generation, and recruiter chat.
LangGraph	Where: backend/orchestration/hr_graph.py Why: Coordinates multi-step HR workflows using graph-based routing and state transitions.
LangChain dependency	Where: backend/requirements.txt Why: Included as part of the AI/agent workflow stack.
Agent modules	Where: backend/agents/*.py Why: Separate AI responsibilities into JD, resume, matching, ranking, summary, interview, recruiter, and router agents.
Workflow modules	Where: backend/workflows/*.py Why: Encapsulate screening, recruiter chat, and application workflows.
Tool gateway	Where: backend/tools/tool_gateway.py Why: Controls agent access to tools and applies policy checks before actions.
Security policy	Where: backend/security/policy.py and backend/security/context_safety.py Why: Adds tool-use guardrails, prompt-injection checks, and safer agent execution.

File Upload and Parsing

Where each technology is used and why

PyPDF2	Where: backend/app/text_extractor.py Why: Extracts PDF text as one parser fallback.
PyMuPDF	Where: backend/app/text_extractor.py Why: Extracts PDF text through fitz, useful for complex PDFs.
pdfplumber	Where: backend/app/text_extractor.py Why: Extracts layout-sensitive PDF text as another fallback.
python-docx	Where: backend/app/text_extractor.py Why: Extracts text from .docx resumes and job descriptions.
secure_filename	Where: backend/services/jd_service.py and backend/services/matching_service.py Why: Sanitizes uploaded filenames before saving files.
Static uploads	Where: backend/static/uploads Why: Stores uploaded resumes and job description files.

Communication, Auth, and Tooling

Where each technology is used and why

smtplib / EmailMessage	Where: backend/services/interview_service.py Why: Sends interview emails, follow-ups, and cancellations through SMTP.
Twilio	Where: backend/services/interview_service.py and backend/requirements.txt Why: Supports SMS/text notifications when configured.
Custom session token	Where: frontend/src/api.js, backend/services/auth_service.py, backend/database.py Why: Stores a frontend token and validates it against MongoDB.
Protected React routes	Where: frontend/src/App.jsx Why: Redirects unauthenticated users to /login.
Audit logs	Where: backend/database.py and backend/routes/audit_routes.py Why: Records important actions with hash-chain integrity checks.
npm/concurrently/cross-env	Where: RecruitmentAssist/package.json Why: Runs Flask and React together during development.
requirements.txt / package-lock.json	Where: backend/requirements.txt and package-lock files Why: Locks and documents backend/frontend dependencies.

Feature-Wise Stack Mapping

Main features and the technologies behind them

Login and authentication	Files: Login.jsx, auth_routes.py, auth_service.py, database.py Tech: React, Flask, MongoDB, Werkzeug hashing, session tokens
Dashboard	Files: Dashboard.jsx, dashboard_routes.py, report_service.py Tech: React, Flatpickr, Flask, MongoDB queries
Job descriptions	Files: JdList.jsx, JdCreate.jsx, JdDetails.jsx, jd_routes.py, jd_service.py Tech: React, Flask, file upload, PDF/DOCX parsing, OpenAI, MongoDB
Candidate management	Files: Candidates.jsx, CandidateProfile.jsx, candidate_routes.py, candidate_service.py Tech: React, Flask, MongoDB, resume parsing
Resume/JD comparison	Files: Comparison.jsx, matching_service.py, screening_workflow.py Tech: React multipart upload, Flask, OpenAI, agent workflow, MongoDB
Interviews	Files: Interviews.jsx, interview_routes.py, interview_service.py Tech: React, Flask, MongoDB, SMTP, Twilio, OpenAI drafts
Reports and insights	Files: Reports.jsx, report_routes.py, report_service.py Tech: React, Flask, MongoDB reporting queries
Recruiter chat	Files: FloatingRecruiterChat.jsx, recruiter_chat_workflow.py, recruiter_agent.py Tech: React, Flask, OpenAI, tool gateway, MongoDB
Agent orchestration	Files: hr_graph.py, main_orchestrator.py, backend/agents/*.py Tech: LangGraph, OpenAI, Flask, MongoDB, security policy

Why This Stack Fits

Summary

React fits because the application has many interactive screens, protected routes, upload flows, forms, modals, dashboards, and recruiter workflows.

Flask fits because the backend needs lightweight API routing, file upload handling, session handling, and direct integration with Python AI and document-processing libraries.

MongoDB fits because recruitment data is flexible: resumes, parsed profiles, job descriptions, comparisons, interviews, audit logs, and agent outputs can vary in shape.

OpenAI fits because the core product depends on natural language understanding: extracting resume/JD details, comparing profiles, creating summaries, generating interview messages, and powering recruiter chat.

LangGraph fits because screening and recruiter assistance are multi-step workflows that need routing, state management, and controlled agent coordination.

PDF/DOCX parsing, SMTP, and Twilio support real recruiter workflows: uploaded documents, email communication, and optional text notifications.

Final Stack

React frontend + Flask backend + MongoDB database + OpenAI AI layer + LangGraph workflow orchestration + PDF/DOCX parsing + SMTP/Twilio communication + custom authentication and audit logging.